

Coding & Solution

Date	
Team ID	xxxxxxx
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Coding & Solution

This section summarizes the implemented code and overall solution for EduGenie.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Arumparithi42/EduGenie-AI
Programming Language(s)	Python 3.10+, JavaScript, HTML5, CSS3
Framework(s) Used	FastAPI, Uvicorn, Jinja2, Pydantic; Google Gen AI SDK (google-genai); optional Hugging Face transformers + PyTorch
Key Features Implemented	1. Q&A with Gemini (GET /qa) 2. Concept explanation - local LaMini-Flan-T5 or Gemini (POST /explain) 3. Quiz generation with JSON validation + interactive scoring (POST /quiz) 4. Summarization (POST /summarize) 5. Personalized learning path (GET /learn/recommendations) 6. Responsive UI, error handling, /health, /docs, automated tests
Pending / Incomplete Features	None of the planned features are pending. Future scope: user accounts & progress tracking, voice input, multilingual UI, mobile app, image/PDF input.
Setup / Run Instructions	1. python -m venv .venv and activate it 2. pip install -r EduGenie/requirements.txt 3. Copy EduGenie/.env.example to EduGenie/.env and set GEMINI_API_KEY 4. cd EduGenie then uvicorn main:app --reload 5. Open http://127.0.0.1:8000 (tests: pytest -v)

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Core Code Excerpt - shared Gemini call (gemini_client.py)

```
def generate(prompt: str, *, json_mode: bool = False, temperature: float | None = None) -> str:
    """Send `prompt` to Gemini and return the response text.

    `json_mode=True` asks Gemini to reply with raw JSON (no Markdown).
    Temporary errors (503 "high demand", 429 rate limits, 5xx, network errors) are retried
    with exponential backoff, then the fallback models are tried in order.
    Raises GeminiNotConfiguredError / GeminiBusyError / GeminiError on failure.
    """
    api_key = get_api_key()
    if api_key is None:
        raise GeminiNotConfiguredError(
            "Gemini API key is not configured. Copy EduGenie/.env.example to "
            "EduGenie/.env and set GEMINI_API_KEY, then restart the server."
        )

    from google.genai import types

    config = types.GenerateContentConfig(
        temperature=temperature,
        response_mime_type="application/json" if json_mode else None,
    )

    primary = get_model_name()
    models = [primary] + get_fallback_models()
    retries = get_max_retries()
    last_exc: Exception | None = None

    for model in models:
        for attempt in range(retries + 1):
            try:
                response = _client(api_key).models.generate_content(model=model, contents=prompt,
config=config)
            except Exception as exc: # network errors, invalid key, quota, overload, unknown
model ...
                last_exc = exc
                code = _error_code(exc)
                if _is_retryable(exc):
                    if attempt < retries:
                        delay = RETRY_BASE_DELAY * (2 ** attempt) + random.uniform(0, 1)
                        logger.warning("Gemini %s busy (%s); retry %d/%d in %.1fs", model, code or
exc, attempt + 1, retries, delay)
                        _sleep(delay)
                        continue
                    logger.warning("Gemini %s still busy after %d retries", model, retries)
                    break # try the next (fallback) model
                if code == 404: # model name not available for this key - try the next model
                    logger.warning("Gemini model %s not found; trying the next model", model)
                    break
                # Not temporary (invalid API key, bad request, ...): retrying will not help.
                logger.error("Gemini request failed: %s", exc)
                raise GeminiError(f"Gemini request failed: {_error_message(exc)}") from exc

            if model != primary:
                logger.warning("Answered with fallback model %s because %s was unavailable",
model, primary)
                text = _extract_text(response)
                if not text:
                    raise GeminiError(
                        "Gemini returned an empty response (it may have been blocked by safety
filters). "
                        "Try rephrasing your input."
                    )
                return text.strip()

    if last_exc is not None and _error_code(last_exc) == 404:
        raise GeminiError(
            f"Gemini model '{primary}' was not found for this API key. Set GEMINI_MODEL in "
            f"EduGenie/.env to a model listed in Google AI Studio. ({_error_message(last_exc)})"
        ) from last_exc
```

```
    tried = ", ".join(models)
    raise GeminiBusyError(
        "Gemini is very busy right now (high demand or rate limit). EduGenie retried automatically "
        f"and also tried the backup models ({tried}). Please wait a minute and try again. "
        f"Details: {_error_message(last_exc)}"
    ) from last_exc
```

Additional Notes / Comments:

- The project document used google-generativeai with gemini-1.5-pro; both are deprecated/retired, so EduGenie uses the current google-genai SDK with a configurable model (default gemini-flash-latest).
- The API key is stored in a git-ignored .env file instead of being hard-coded.
- 40 automated tests (pytest, Gemini mocked) cover every endpoint, validation and error path.